7/10/26

EXP-1

caesar cipher

Name : A. Midhuna

Register No : 192521302

Aim:

To write a c program to implement the caesar cipher where each letter is replaced.

Algorithm:

* Read the plaintext and the key value from the user.
* check each character of the plaintext.
* For each alphabet shift it by K position using modulo 26.
* keep non-alphabetic character unchanged
* Display the encrypted ciphertext.

pseudocode:

```
For each ch in plaintext
    if ch in uppercase
        ch = ((ch - 'A' + K) MOD 26) + 'A'
    else if ch in lowercase
        ch = ((ch - 'a' + K) MOD 26) + 'a'
    end if
end for
Display ciphertext
END
```

INPUT:	output:
HELLO	KHOOR
3	

Result:

Thus the c program implemented success for caesar cipher.

Exp 2 Substitution cipher

2) Aim: To write c program implement a monoalphabetic substitution cipher where each plaintext letter is mapped uniquely.

Algorithm:

* Read the plaintext and substitution key.
* check each character of the plaintext
* replace each alphabet with its corresponding key character.
* keep space and special character unchanged
* Display the encrypted message.

pseudocode.

```
if (plaintext [i] >= 'A' && plaintext [i] <= 'z')
{ pos = plaintext [i] - 'A';
  ciphertext [i] = toupper (key [pos]);
}
else if (plaintext [i] >= 'a' && plaintext [i] <= 'z')
{
  pos = plaintext [i] - 'a';
  ciphertext [i] = tolower (key [pos]);
}
else {
ciphertext [i] = plaintext [i];
}
```

INPUT:

plaintext: HELLO

Key: QWERTYUIOPASDFGHJKLZXCVBNM

OUTPUT:

Ciphertext: ITSSG

Result:

Thus the Monoalphabetic substitution cipher was successfully implemented in c.

3. Aim:

To write c program implement the playfair cipher which encrypts plaintext two letter at a time using 5x5 matrix

Algorithm

* Read the keyword and plaintext
* construct 5x5 matrix using keyword removing duplicate letter combining I/J
* Divide the plaintext into pair of letters and insert x when needed.
* Encrypt each pair of letter and insert x.
* Display the encrypted ciphertext.

Pseudocode:

```
for (i=0; text[i]; i+=2){
  int a = toupper(text[i]) - 'A';
  int b = toupper(text[i+1]) - 'A';
  int r1 = a/5, c1 = a%5, r2 = b/5, c2 = b%5;
  if (r1 == r2)
    printf("%c%c", m[r1][(c1+1)%5],
                   m[r2][(c2+1)%5]);
  else if (c1 == c2)
    printf("%c%c", m[(r1+1)%5][c1],
                   m[(r2+1)%5][c2]);
  else
    printf("%c%c", m[r1][c2], m[r2][c1]);
}
```

INPUT:

Enter key: MONARCHY

Enter plaintext: INSTRUMENTS

OUTPUT:

Encrypted text: GIATLMZCLREIXA

Results:

Thus, The playfair cipher encryption algorithm was implemented successfully.

Exp 4: polyalphabetic & monoalphabetic substitution

4. Aim:

To implement the polyalphabetic substitution cipher using a key encrypt the plaintext.

Algorithm:

* Start the program
* Get the plaintext and key, from the user
* Repeat key until it matches plaintext length.
* Encrypt each letter using formula $C = (p+k) \bmod 26$
* Display the encrypted ciphertext.
* Stop the program

Pseudocode:

```
n = Strlen(p);
l = Strlen(k);
for(i=0; i<n; i++)
    printf("%c", (toupper(p[i]) - 'A' +
        toupper(k[i%l]) - 'A') % 26 + 'A');
```

```
return 0;
}
```

INPUT

Enter plaintext: HELLO

Enter key: KEY

output

ciphertext: RIJVS

Results:

The polyalphabetic substitution cipher was implemented successfully plaintext was encrypted using given key.

5. Exp 5: caesar cipher

Aim:

To implement the Affine caesar cipher using the encryption formula $C = (ap+b) \bmod 26$ & identify restrictions keys a and b.

Algorithm:

* Start the program
* Get plaintext and keys a and b from user
* check that a relatively prime to 26.
* convert each plaintext letter into number from 0 to 25.
* Encrypt using $C = (ap+b) \bmod 26$
* Display the ciphertext
* stop the program.

Pseudocode:

```
if (a%2==0||a%13==0){
    printf("Invalid key a");
    return 0;
}
for(i=0; p[i]; i++){
```

```
int x = toupper(p[i]) - 'A';
printf("%c", (a*x + b) % 26 + 'A');
}
return 0; }
```

Input:

Enter plaintext: HELLO

Enter a and b: 5 8

output

ciphertext: RCLLA

Results:

Affine caesar cipher encryption was implemented successfully.

Exp 6: Affine cipher

6. Aim:

To break an Affine cipher using frequency analysis where frequent ciphertext letter B & second most frequent U.

Algorithm:

* Start the program.
* Read the ciphertext.
* Assume most frequent plaintext letters E and T.
* map ciphertext B and U. assumed plaintext letters.
* Find possible keys a and b Affine cipher formula
* Decrypt the ciphertext using $P = a^{-1}(C-b) \bmod 26$.
* Display the decrypted plaintext
* Stop.

Pseudocode:

```
for(i=1; i<26; i++)
  if((a*i)%26==1) inv=i;
if(inv==0){
  Printf("Invalid key");
for(i=0; c[i]; i++){
   j=c[i]-'A';
  printf("%c",((inv*(j-b+26))%26)+'A');
}
return 0;
}
```

Input:

most frequent ciphertext letter: B

Second most frequent ciphertext letter: U

Output:

possible plaintext: E and T

Affine cipher Depends on ciphertext

Decrypted text Depends on the ciphertext

Result

The frequency analysis used identify possible mapping determine Affine cipher key.

7 Exp 7: Simple substitution

Aim:

To implement a simple substitution cipher to analyze and decrypt given ciphertext.

Algorithm

* Start the program
* Read the given ciphertext
* Analyze frequency each symbol
* Replace ciphertext symbols corresponding plaintext

letters using substitution key.

* Display decrypted plaintext.
* Stop the program

Pseudocode:

```
for(i=0; c[i]; i++) {
  if (c[i]>='A' && c[i]<='z') {
    for(j=0; j<26; j++)
      if (c[i]==key[j])
        printf("%c", p[j]);
  } else
      printf("%c", c[i]);
  return 0;
}
```

Input:

53‡‡305))6*;4826)4‡.)

Output:

Decrypted plaintext (requires substitution key)

Result:

The simple substitution cipher decryption program was outlined analyze decrypt given ciphertext.